

Milestone 2 Report

SCIPER 392412 · 373176 · 398388

1 Introduction

The Gaming and Mental Health Dataset contains 2,000 samples exploring gaming habits and their relation to mental health outcomes. Following our setup in Milestone 1, we utilize 1,600 samples for training and 400 samples for testing.

The project consists of two core machine learning tasks:

1. **Regression:** Predicting the continuous addiction score, ranging from 0 to 10.
2. **Classification:** Classifying the general addiction level into one of three categories: Low, Medium, or High.

In this Milestone, we transition from linear and instance-based baselines to more complex, parametric models. Specifically, we implement from scratch a Multi-Layer Perceptron (MLP) for both tasks and K-Means Clustering for classification.

2 Methods

We prepared the data by applying z-score normalization using the training set’s mean and standard deviation. To perform robust hyperparameter tuning and avoid test-set data leakage, we split the 1,600 training samples into an 80/20 train/validation split (1,280 training, 320 validation).

2.1 Multi-Layer Perceptron (MLP)

Our custom MLP supports arbitrary hidden layers and layer widths. Weights are initialized using a normal distribution scaled by $1/\sqrt{I}$, where I is the layer’s input dimension, and biases are initialized to zero.

For a layer l , the forward pass computes:

$$\mathbf{A}^{(l)} = \mathbf{Z}^{(l-1)}\mathbf{W}^{(l)\top} + \mathbf{b}^{(l)} \quad (1)$$

$$\mathbf{Z}^{(l)} = \sigma^{(l)}(\mathbf{A}^{(l)}) \quad (2)$$

where $\mathbf{Z}^{(0)} = \mathbf{X}$ is the input feature matrix. We implement Rectified Linear Unit (ReLU) and Sigmoid activations.

For training, backpropagation computes gradients layer-by-layer backwards using the chain rule. Weights and biases are updated using Mini-Batch Gradient Descent (batch size = 32) with a configurable learning rate (lr). Shuffling is executed at the start of each epoch. We use two loss functions:

- **Mean Squared Error (MSE):** Used for regression, measuring squared distance between predictions and targets.
- **Categorical Cross-Entropy (CE):** Used for multi-class classification, incorporating a small offset ($\epsilon = 10^{-9}$) to ensure numerical stability when computing $\log(y_{\text{pred}})$.

2.2 K-Means Clustering

K-Means groups the N normalized features into K clusters. Initial centroids are randomly picked from the dataset. Datapoints are iteratively assigned to the closest centroid based on Euclidean distance:

$$d(\mathbf{x}_i, \mathbf{c}_k) = \|\mathbf{x}_i - \mathbf{c}_k\|_2 \quad (3)$$

Centroids are updated by taking the mean of all points assigned to them. Empty clusters are handled by re-assigning the centroid to a random point.

To use K-Means for classification, we assign a label to each centroid based on the majority vote of the true training labels belonging to its cluster. For validation or testing, a new sample is assigned to its nearest centroid, adopting that centroid’s voted label.

3 Experiments and Results

3.1 K-Means Clustering

We evaluated K-Means classification for $K \in [1, 20]$ on the validation set.

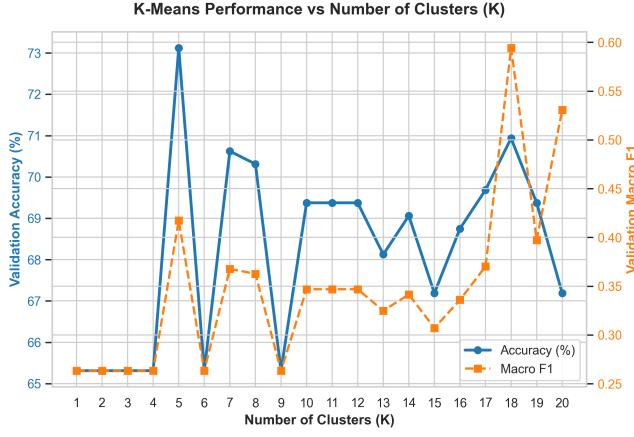


Figure 1: K-Means Performance Across Cluster Sizes (K)

The best accuracy was achieved at $K = 5$, reaching 73.12% with a Macro F1 score of 0.4172. Interestingly, while $K = 18$ yielded a slightly lower accuracy of 70.94%, it achieved a significantly higher Macro F1 score of 0.5943. This is because higher cluster counts allow the model to isolate and represent minority classes, preventing them from being overshadowed by majorities during voting.

3.2 MLP Classification

We tuned the MLP classifier (hidden size = 64, activation = ReLU/Sigmoid) across learning rates $lr \in [10^{-4}, 0.5]$ and epochs $\in [50, 500]$.

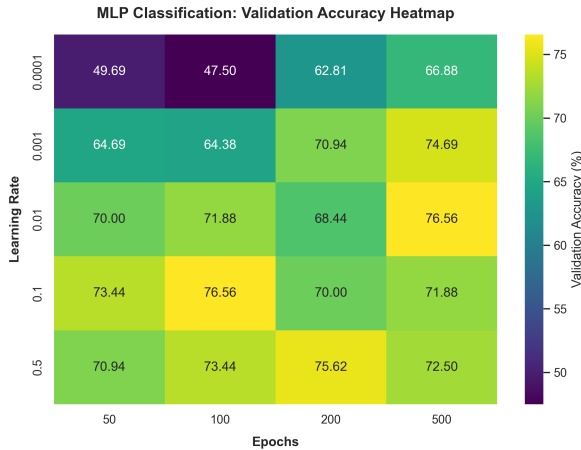


Figure 2: MLP Classification Accuracy Heatmap

The best performance was achieved at $lr = 0.01$ with 500 epochs, reaching a validation accuracy of 76.56% and F1 of 0.4541. We observed that small learning rates ($lr = 10^{-4}$) converged too slowly,

while large rates ($lr = 0.5$) led to gradient instability and sub-optimal validation accuracy (72.50% at 500 epochs).

3.3 MLP Regression

We tuned the MLP regressor (hidden size = 64, activation = ReLU/ReLU) on the continuous addition score task.

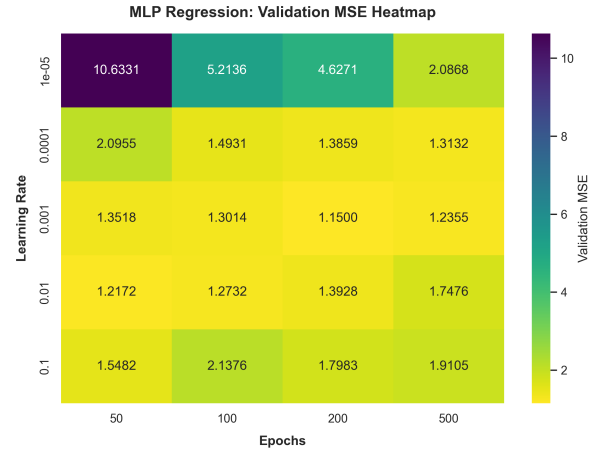


Figure 3: MLP Regression MSE Heatmap (Lower is Better)

The baseline parameters ($lr = 10^{-5}$, 100 epochs) got a validation MSE of 5.2136. Tuning learning rate to $lr = 0.001$ with 200 epochs significantly improved convergence, yielding our best validation MSE of 1.1500. Beyond $lr = 0.01$, validation MSE worsened (1.7476 at 500 epochs), indicating overfitting or optimization divergence.

3.4 Summary of All Results

The table below summarizes the final performance of our optimized Milestone 2 models.

Task	Model (Parameters)	Performance
Classification	K-Means ($K = 5$)	Acc: 73.12%, F1: 0.4172
Classification	K-Means ($K = 18$)	Acc: 70.94%, F1: 0.5943
Classification	MLP ($lr = 0.01$, ep=500)	Acc: 76.56%, F1: 0.4541
Regression	MLP ($lr = 0.001$, ep=200)	MSE: 1.1500

Table 1: Summary of best results for Milestone 2 models

4 Extensions

To enhance our machine learning pipeline, we implemented three key extensions:

- **Categorical Cross-Entropy from Scratch:** Built categorical cross-entropy loss and its gradient vectorization from scratch rather than using standard binary loss, enabling multi-class classification.
- **Deep MLP Architectures:** Leveraged our general MLP constructor to test deep architectures. We explored a 2-hidden-layer network of dimensions $(D, 64, 32, C)$. However, we found that it marginally degraded validation accuracy compared to a single-hidden-layer network, likely due to overfitting on 1,280 training samples.
- **Mini-Batch Data Shuffling:** Shuffled training indices before every epoch. This randomizes batch compositions, preventing the network from getting stuck in gradient cycles.

5 Discussion and Conclusion

We successfully implemented, tuned, and evaluated both MLP and K-Means models.

For regression, our tuned MLP model obtained an MSE of 1.1500, showing that a non-linear network can successfully fit the complex patterns of gaming habits.

For classification, the MLP classifier outperformed K-Means by achieving 76.56% accuracy compared to K-Means' 73.12%. While K-Means works well as an unsupervised clustering baseline, the supervised MLP classifier is better suited since it optimizes directly for class boundaries.

In conclusion, parametric models offer a substantial performance improvement over simpler baselines, but hyperparameter tuning is critical to prevent slow convergence or numerical instability.